

数据结构与算法 K02 课堂练习

1900011604 张植竣 指导老师: 陈斌

北京大学物理学院天文学系

2020 年 3 月 3 日

1 验证 `list.insert(0, item)` 的时间复杂度为 $O(n)$

代码如下:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize as opt
from timeit import Timer

...

计时实验程序1
list.insert(0, item): 添加数据项成为列表的第一个元素
...

def test1():
    return l.insert(0, -1)

t1 = Timer('test1()', 'from __main__ import test1')
x_list1 = []
y_list1 = []
for n in range(100000, 1100000, 100000): # 10个数据点
    l = list(range(n))
    x_list1.append(n)
    y_list1.append(t1.timeit(number = 1000))
for i in range(len(x_list1)):
    x_list1[i] /= 1000000
    print('%.7f'%(y_list1[i]))

# 线性拟合数据点

def f1(n, k, b):
    return k * n + b
```

```

fita, fitb = opt.curve_fit(f1, x_list1, y_list1)
print('\nk=' + str(fita[0]))
print('b=' + str(fita[1]))
print('t=%.2fn+%.2f\n'%(fita[0], fita[1]))

# 绘图

xmax1 = x_list1[-1] # x轴的上界
dx1 = x_list1[-1] / 5 # x轴的单位长度
ymax1 = float('%0.4f'%max(y_list1)) # y轴的上界
dy1 = ymax1 / 5 # y轴的单位长度
x1 = np.arange(0, xmax1 + dx1, 0.01) # 绘直线用
plt.figure(figsize = (16, 10))
plt.scatter(x_list1, y_list1, color = 'g', marker = '+')
plt.plot(x1, f1(x1, fita[0], fita[1]), color = 'r')
plt.title('K02-1', fontsize = 24)
plt.xlabel("n(*10^6)", fontsize = 16)
plt.ylabel("time(s)", fontsize = 16)
plt.xlim((0, xmax1 + dx1))
plt.ylim((0, ymax1+dy1))
xaxis = np.arange(0, xmax1 + dx1, dx1)
yaxis = np.arange(0, 2 * ymax1 + 4 * dy1, dy1)
plt.xticks(xaxis)
plt.yticks(yaxis)

plt.show()

```

输出结果:

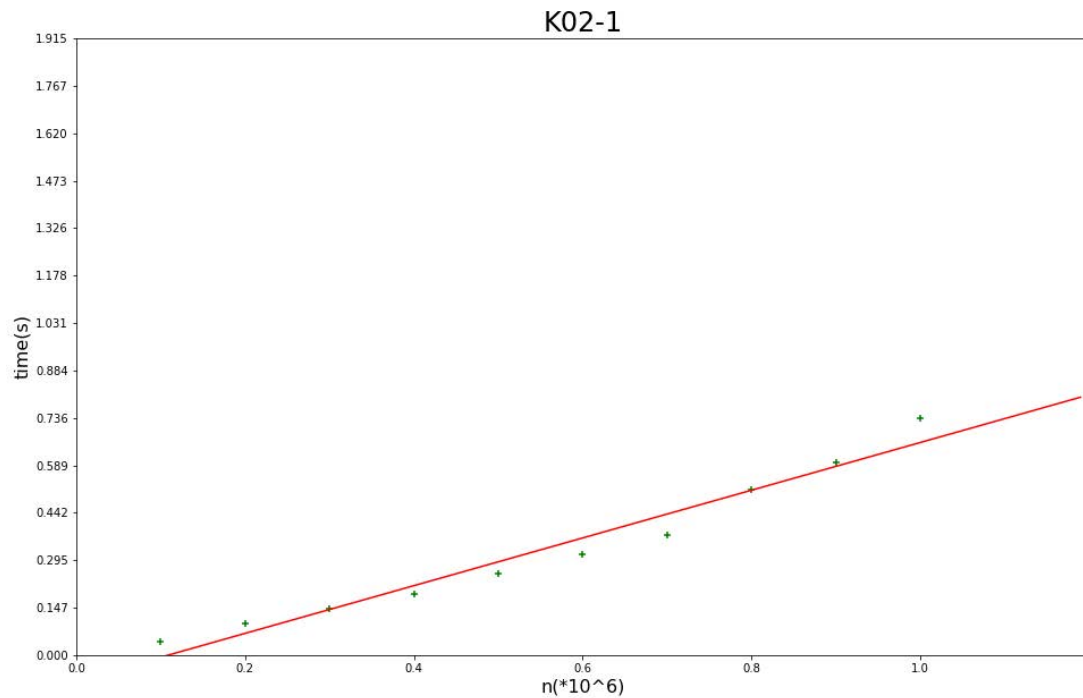
```

0.0419501
0.0979203
0.1435118
0.1903678
0.2540433
0.3134986
0.3717452
0.5151925
0.5997339
0.7364233

k=0.7409057582920955
b=-0.08105948706332189
t=0.74n+-0.08

```

绘制图像如下：



实验结果分析：

可以看到，运行 `list.append(item)` 函数 10^6 次时，数据点集中分布在直线 $t = 0.74n - 0.08$ 附近，拟合出的斜率 $k \approx 0.74$ （考虑到重复运行了 10^3 次，实际上 $k \approx 7.4 \times 10^{-4}$ ），说明 `list` 取值所需时间与问题的规模（即列表长度 n ）成线性关系，时间复杂度为 $O(n)$ 。

2 验证 `list.append(item)` 的时间复杂度为 $O(1)$

代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize as opt
from timeit import Timer

...

计时实验程序2
list.append(item): 添加数据项成为列表的最后一个元素
...

def test2():
    return l.append(-1)
```

```
t2 = Timer('test2()', 'from __main__ import test2')
x_list2 = []
y_list2 = []
for n in range(100000, 1100000, 100000): #10个数据点
    l = list(range(n))
    x_list2.append(n)
    y_list2.append(t2.timeit(number = 100000))
for i in range(len(x_list2)):
    x_list2[i] /= 1000000
    print('%.7f'%(y_list2[i]))

# 线性拟合数据点

def f2(n,k,b):
    return k * n + b

fita, fitb = opt.curve_fit(f2, x_list2, y_list2)
print('\nk=' + str(fita[0]))
print('b=' + str(fita[1]))
print('t=%.2fn+%.2f\n'%(fita[0], fita[1]))

# 绘图

xmax2 = x_list2[-1] # x轴的上界
dx2 = x_list2[-1] / 5 # x轴的单位长度
ymax2 = float('%.4f'%max(y_list2)) # y轴的上界
dy2 = ymax2 / 5 # y轴的单位长度
x2 = np.arange(0, xmax2 + dx2, 0.01) # 绘直线用
plt.figure(figsize = (16, 10))
plt.scatter(x_list2, y_list2, color='g', marker='+')
plt.plot(x2, f2(x2, fita[0], fita[1]), color='r')
plt.title('K02-2', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(ms)", fontsize=16)
plt.xlim((0, xmax2 + dx2))
plt.ylim((0,ymax2+dy2))
xaxis = np.arange(0, xmax2 + dx2, dx2)
yaxis = np.arange(0, 2 * ymax2 + 4 * dy2, dy2)
plt.xticks(xaxis)
plt.yticks(yaxis)

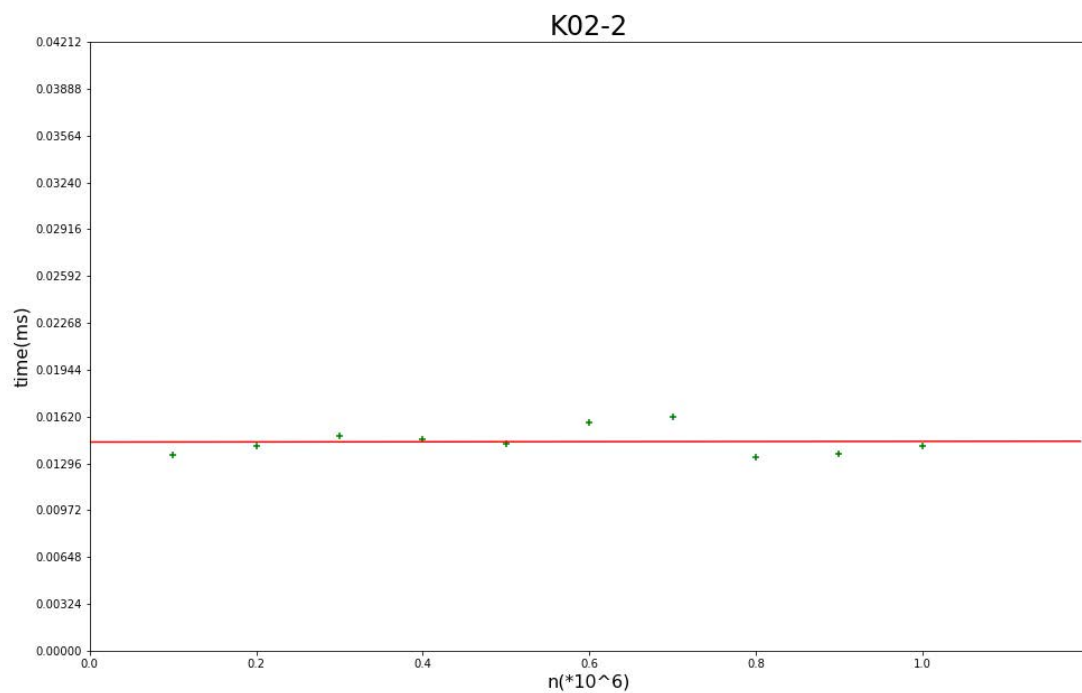
plt.show()
```

输出结果：

```
0.0135574
0.0141594
0.0148340
0.0146109
0.0142911
0.0157744
0.0161587
0.0134244
0.0136601
0.0141383
```

```
k=4.919324249107837e-05
b=0.01443381371663042
t=0.00n+0.01
```

绘制图像如下：



实验结果分析：

可以看到，运行 `list.append(item)` 函数 10^5 次时，数据点集中分布在水平线 $t = 0.0144s$ 附近，拟合出的斜率 k 几乎为 0（考虑到重复运行了 10^5 次，实际上 $k \approx 4.92 \times 10^{-10}$ ），说明 `list` 取值所需时间与问题的规模（即列表长度 n ）无关，时间复杂度为 $O(1)$ 。